EKAGRA

AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

Problem Statement ID	SIH26145
Problem Statement	AI-Based Detection of Cyber Threats in Unidirectional IP Traffic
Organisation	National Technical Research Organisation (NTRO)
Theme	Blockchain and Cybersecurity
PS Category	Software
Team	AlgoRhythms
Document type	Complete technical, functional and presentation reference
Status of figures	All measurements read from the repository's own results files at authoring time
Companion project	KALACHAKRA (SIH26153) - shares roughly 60-70% of this data foundation

This document is written to be self-sufficient. A new teammate, an evaluator, a developer joining the project, a designer, or another AI model should be able to understand EKAGRA completely from this document alone - what it is, why it exists, how every part works, what has actually been measured, where it fails, and how to present and demonstrate it.

Table of Contents

1. Executive Summary

1.1 What the project is

EKAGRA is a **network threat-detection sensor engineered for a one-way tap** - a data diode or passive mirror that carries traffic into a secure enclave but provides no path back out. It ingests packets or flow records, assembles them into flows, maintains streaming per-host state over 60-second windows, scores six classes of threat, calibrates each score so the system can say *I do not know* rather than guess, and writes every alert into a tamper-evident hash-chained evidence log. It ships with a browser console that runs with no network access at all.

1.2 The problem it solves

Published intrusion-detection systems and the datasets they are trained on assume **bidirectional visibility**: that the sensor can see both halves of a conversation, and often that it can make outbound calls to threat-intelligence or reputation services. A monitoring enclave behind a data diode has neither. Deploy a conventionally-trained detector there and its accuracy collapses - which this project measured rather than assumed.

1.3 Why it matters and who experiences it

India's Critical Information Infrastructure - defence, power, banking, telecom - increasingly isolates sensitive segments behind unidirectional gateways precisely so that nothing can be sent *in*. That control creates the blind spot this project addresses: the security monitoring that protects the enclave is itself degraded by the control protecting the enclave. The direct stakeholders are NTRO, NCIIPC-designated operators, and the analysts staffing such enclaves.

1.4 How the solution addresses it

The project's central finding is that the loss is **not primarily about direction**. It is about **per-host state**. A per-flow feature extractor pointed at a one-way tap loses roughly 0.29 binary-F1. The same tap, with a sensor that maintains windowed per-host aggregates - fan-out, source-IP entropy, peer concentration, arrival periodicity, volume against a per-host baseline - loses essentially nothing. EKAGRA is the sensor built to that measured profile.

1.5 What makes the approach valuable

- **It is a measurement, not a claim.** The visibility profile of a one-way tap was measured on 877,801 real flows before anything was built to exploit it.
- **Every capability claim ships with the control that narrows it.** The project's own control refuted its original headline and that refutation is published, not buried.
- **It runs where it must run.** Zero outbound connections in the detection path, enforced by a continuous-integration test that fails the build if any component opens a socket. The console has no CDN dependency.
- **It abstains.** Conformal calibration gives a per-class coverage guarantee and an explicit ABSTAIN decision, so a low-evidence case is reported as such rather than guessed.
- **Every alert is provable.** Alerts carry packet indices, the feature vector, a model hash and a SHA-256 chain link to the previous alert.

1.6 Overall impact

For an enclave operator EKAGRA removes a recurring commercial licence and an egress path at the same time. Commercial network detection and response is licensed per protected IP; removing the reputation-feed dependency was additionally measured to *improve* detection on this corpus, so the constraint costs nothing here and closes a hole.

The 30-second answer to "What is your project?"

"Secure networks are increasingly monitored through a data diode - the sensor sees traffic but has no way to send anything back. Every published intrusion detector assumes it can see both directions and phone a threat-intelligence service. We measured what that costs on 877,801 real flows: an off-the-shelf detector drops from 0.98 to 0.69 F1. Then we found why - it is not the lost direction, it is the lost per-host state. Our sensor keeps 60-second per-host statistics and gets back to 0.981 on the same one-way tap. It runs air-gapped, it abstains when the evidence is thin, and every alert is hash-chained so you can prove it."

Read this before quoting any number from this document

During authoring, five contradictions were found between the repository's documents, its results files and its shipped presentation. They are catalogued in Section 26.6 with a ruling on which source is authoritative in each case. Two of them affect numbers currently on the submission slides. Do not quote a figure from memory - Section 26 lists the measured values and the file each came from.

2. Problem Statement

2.1 The problem as the client states it

Problem statement SIH26145 is issued by the National Technical Research Organisation and describes a data diode or passive mirror deployment. The repository records three constraints taken from the PS text, each of which is enforced somewhere in the code rather than merely documented.

Constraint	What the PS requires	How the project enforces it
Read-only ingest	Any design assuming a return path, a live query to the source, or an inline block is out of scope.	No outbound sockets in the detection path. A test in `tests/test_readonly_constraint.py` fails the build if one is opened.
No payload decryption	TLS and QUIC sessions must be analysed from metadata only.	Detectors consume handshake metadata, packet sizes and timings. Payload bytes never reach feature code.
Streaming, not batch	Traffic must be processed incrementally with bounded alert latency.	Single-pass iterator, O(1)-memory sketches, windowed emit. No global sort and no full-dataset pass.

Table 1. The three PS constraints and their enforcement points.

2.2 Why existing approaches are insufficient

Three distinct classes of existing solution fail here, and they fail for different reasons. This distinction matters because it is what makes the problem a real gap rather than a procurement question.

Existing approach	What it requires	Why it fails behind a diode
Signature and script IDS (Suricata, Zeek)	Rule and signature feeds; both directions of a flow for most rules.	The rule-update path is itself an egress path the diode forbids. Rules keyed on server responses cannot fire when responses are unobservable.
Commercial NDR (Darktrace, Vectra and similar)	Cloud analytics callback; licensing per protected IP.	Requires a callback the enclave cannot provide, adds a recurring licence, and moves telemetry outside the enclave and outside India.
Data-diode vendors (Owl, Waterfall and similar)	Hardware unidirectional link.	They move data one way correctly - they do not perform threat detection on it. They are the constraint, not the detector.
Academic NIDS trained on public corpora	Bidirectional flow records (CIC-IDS2017, CSE-CIC-IDS2018, UNSW-NB15 all ship bidirectional features).	Trained on visibility they will not have at deployment. Measured cost on this corpus: binary F1 0.9793 falls to 0.6903.

Table 2. Why each class of existing solution does not transfer to a one-way tap.

2.3 Real-world consequences

An enclave monitored by a detector trained under the wrong visibility assumption produces two failure modes at once: it misses genuine intrusions because the signals it relies on are unobservable, and it produces false alarms because features it was trained on now take degenerate values. The operator has no way to distinguish the two without a controlled measurement - which is what this project supplies.

2.4 The specific gap addressed

The gap is not "a better classifier". It is the absence of a **measured visibility profile** for a one-way tap, and of a sensor designed against that profile. The project's contribution is the procedure - characterise what the tap can observe, then train and build under exactly that - together with the sensor produced by applying it.

2.5 Threat classes and the signal each loses

The problem statement names six threat classes. For five of the six, the primary classical signal is on the reverse channel and is therefore unobservable. This table is the design rationale for the entire feature set.

Threat class	Classic bidirectional signal	Lost?	Forward-side replacement used
Volumetric / protocol DDoS	SYN with no returning SYN-ACK	Yes	Source-IP entropy (HyperLogLog and Count-Min), SYN rate per destination, spoofed-source dispersion
C2 beaconing	Request/response round-trip regularity	Partly	Inter-arrival periodicity and jitter on forward packets only; destination concentration
DGA / DNS tunnelling	NXDOMAIN response rate	Yes	Query-name character entropy, n-gram improbability, label length, query-type mix
Malware in encrypted sessions	Server certificate, JA3S	Yes	Client JA4 fingerprint rarity, packet-size sequence, inter-arrival sequence
Reconnaissance / scanning	RST responses from closed ports	Yes	Fan-out cardinality per source over destination ports and hosts, burst structure
Data exfiltration	Outbound-to-inbound byte ratio	Yes	Absolute outbound volume against a per-host decayed baseline, upload burst shape

Table 3. Five of six threat classes lose their primary signal under a one-way tap.

3. Motivation and Relevance

3.1 Why the project was created

It was created to answer SIH26145. The motivation that shaped it, however, came from noticing that the problem statement describes a deployment condition that essentially the entire public NIDS literature trains against the opposite of. That is an unusual situation: a well-studied field whose standard evaluation setup does not match the stated deployment. It makes a measurement valuable in a way that another model architecture would not be.

3.2 Why the problem is relevant now

- **Volume.** CERT-In handled 29.44 lakh cyber incidents in 2025, up from 20.41 lakh in 2024 and 15.92 lakh in 2023 - an 85% rise in two years. Source: CERT-In / PIB, cited on the submission slides.
- **Architecture trend.** Unidirectional gateways are increasingly standard for operational-technology and defence segments, which means the monitoring blind spot they create is becoming more common, not less.
- **Sovereignty.** A detector that calls a foreign cloud service is a policy problem for Critical Information Infrastructure independent of whether it is technically permitted.

3.3 Industry, social and economic relevance

Commercial network detection and response is licensed per protected IP. At published rates in the range of USD 8-22 per protected IP per year, a 2,000-IP enclave carries a recurring cost on the order of Rs 13-37 lakh per year, indefinitely, plus a callback the enclave cannot architecturally provide. EKAGRA's licence cost is zero and its egress requirement is zero. [INFERENCE: the rupee range is a straightforward arithmetic projection from published per-IP list pricing and an assumed 2,000-IP enclave; it is not a quotation and no procurement was performed.]

3.4 Potential real-world applications

- Threat monitoring inside an air-gapped or diode-protected defence enclave.
- Passive monitoring at an ISP or campus SPAN port where only one direction is mirrored.
- Any sensor placement where the return path is unavailable or administratively forbidden.
- As a methodology: characterising the visibility profile of *any* sensor placement before training a detector for it.

4. Proposed Solution

4.1 Core idea

Build the sensor to the tap, not the tap to the sensor. Concretely: (1) measure precisely which features survive a one-way tap; (2) identify which lost capability actually causes the accuracy drop; (3) replace that capability with something computable from forward traffic alone; (4) calibrate the result so it can abstain; (5) make every output provable.

4.2 The finding the design rests on

Central measured result

On the corrected CIC-IDS2017 re-extraction (877,801 flows, temporal split), full visibility scores 0.9793 binary F1. The same model deployed on a one-way tap scores 0.6903. Adding 16 streaming host-window features recovers it to 0.9810 - level with full visibility. Source: `ekagra/backend/results/exp04_verdicts.json`.

4.3 Main components and how they interact

Input to output, the system is a single-pass pipeline. Each stage may only read from the stage before it; nothing may call anything upstream.

Component	Responsibility	Input	Output
Source adapters	Turn a capture or corpus into a uniform packet or flow iterator	pcap / pcapng file, CSV corpus, or the synthetic generator	Packet records: timestamp, 5-tuple, size, flags, TTL, DNS qname, TLS fingerprint
Visibility filter	Model the tap by dropping one direction	Packet stream	Observed packets only
Flow assembler	Group packets into flows with timeouts	Observed packets	Flow records (forward-only state)
Streaming host-window	Maintain per-host aggregates over a tumbling window	Flow records	WindowFeatures per (host, window) cell
Detector heads	Score each threat class independently	WindowFeatures plus per-flow features	Raw per-class scores
Conformal calibration	Convert raw scores into calibrated confidence or ABSTAIN	Raw scores plus a held-out calibration set	Calibrated decision with a coverage guarantee

Component	Responsibility	Input	Output
Evidence bundle	Make each alert provable and tamper-evident	Decision plus contributing packet indices and feature vector	Hash-chained evidence record
Read-only API and console	Present alerts to an analyst	Evidence log	GET-only JSON; a browser console with no network dependency

Table 4. Component responsibilities as an input-processing-output chain.

5. System Architecture

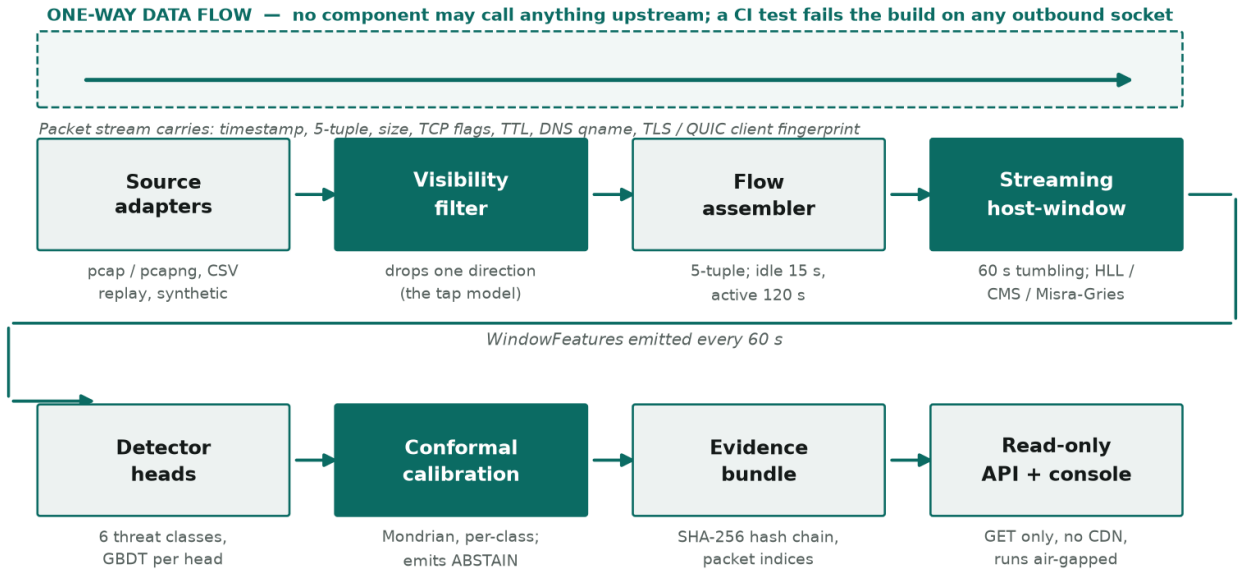

Figure 1. EKAGRA end-to-end architecture. The dashed banner is an enforced property, not a description: a CI test fails the build if any component in the detection path opens an outbound socket.

5.1 Layer by layer

Ingest layer

What it is. Source adapters plus the visibility filter. **Why it exists.** Every downstream stage must see the same record shape whether the input is a live capture, a public corpus, or generated traffic - otherwise a synthetic-versus-real difference would be unattributable to the model. **What flows through it.** Packet records carrying timestamp, 5-tuple, size, TCP flags, TTL, DNS query name and TLS/QUIC client fingerprint.

The packet path is a genuine implementation, not a wrapper: `ingest/pcap.py` (556 lines) reads both pcap and pcapng, decodes IPv4 and IPv6, TCP and UDP, and reassembles TLS ClientHello messages that span TCP segments. `ingest/wire.py` (484 lines) parses DNS query names with compression-pointer safety and computes JA3, JA3S and JA4 fingerprints. `ingest/quic.py` (343 lines) derives QUIC Initial keys from the Destination Connection ID using the published RFC 9001 salt and decrypts the Initial packet to reach the ClientHello inside.

Feature layer

What it is. Per-flow extractors plus the streaming host-window aggregator. **Why it exists.** This layer carries the project's central finding - the host-window aggregates are the component that makes a one-way tap viable. **Key mechanism.** Cardinality and entropy at line rate cannot use dictionaries, so `features/sketches.py` provides Count-Min Sketch, HyperLogLog and exponentially-decayed counters. Estimators start exact and promote to sketches only above 256 distinct values, which is why most features remain bitwise identical to an exact batch computation.

Detection layer

What it is. Six independent heads, one per PS threat class: volumetric, beacon, DGA, scan, exfil, encrypted. **Why independent.** Threat classes co-occur rarely in practice - measured co-occurrence in experiment 09 was 0.082% of test cells - so independent heads are both simpler and more interpretable than a single multiclass model, and each head can declare its own known gaps.

Calibration layer

What it is. Split-conformal prediction over a held-out calibration set, in Mondrian (per-class) mode. **Why it exists.** A raw model score is not a probability and cannot support an operational decision. **What it produces.** A calibrated confidence, a coverage guarantee, and an explicit ABSTAIN decision labelled `INSUFFICIENT_EVIDENCE_UNDER_UNIDIRECTIONAL_CONSTRAINT`. Abstention is a first-class output, not a failure state.

Evidence layer

What it is. A per-alert bundle containing the observed packet indices, the feature vector, a model hash, the decision path, a SHA-256 chain link to the previous alert, and an **HMAC-SHA256** over the bundle's canonical form keyed by a secret the enclave holds. **Why it exists.** In an enclave the analyst cannot re-query the source to check an alert; the alert must carry its own proof. **Why both.** The chain makes editing detectable; the HMAC is what stops a competent attacker simply recomputing every hash forward and producing a consistent forgery. Canonicalisation is deliberate - JSON key order, float formatting and negative zero are all pinned by tests, because a digest over a non-canonical form is not reproducible across languages.

Presentation layer

What it is. A read-only HTTP API and a single-file browser console. **Why it is built this way.** The deployment target has no route to the internet, so a CDN script tag would fail and a build step would mean the artefact running in the enclave is not the artefact in the repository. There is no framework and no bundler.

5.2 What is deliberately not built

Not built	Reason recorded in the repository
Login, RBAC, multi-tenancy	No evaluation credit; days of work.
A mobile application	Not in the problem statement.
Any LLM in the detection path	Latency, non-determinism, and it would break the evidence guarantee.
An inline blocking mode	Explicitly out of scope per the PS.
Six detectors at mediocre quality	Three done properly was judged better than six at 60%.

Table 5. Deliberate scope exclusions and their stated rationale.

6. End-to-End Workflow

The lifecycle below is the actual order of operations. Stage numbers are referenced later by the demonstration script in Section 31.

1. **Packet arrives at the tap.** A source adapter yields a Packet record. For a live capture this is pcap or pcapng decoding; for corpus work it is a CSV row; for development it is the synthetic generator.
2. **Visibility filter applies the tap model.** One direction is dropped. This is simultaneously the research ablation and the production component - in deployment the filter *is* the tap.
3. **Flow assembly.** Packets are grouped by 5-tuple into flows, closed by TCP flags, by a 15-second idle timeout, or by a 120-second active timeout. Only forward-direction state is maintained.

4. **Host-window aggregation.** Each flow updates per-host aggregates in a 60-second tumbling window: fan-out, distinct destination ports, source-IP entropy, peer concentration, inter-arrival statistics, forward bytes and packets against a decayed per-host baseline.
5. **Window closes.** After the window plus a 15-second lateness budget, WindowFeatures are emitted for every (host, window) cell. End-to-end alert latency is therefore 75 seconds.
6. **Detector heads score.** Six heads score the cell independently, each consuming its own declared feature subset.
7. **Conformal calibration.** Scores are compared against per-class calibration quantiles. The output is either a calibrated decision or ABSTAIN.
8. **Severity assignment.** Kill-chain stage impact is combined with a volume-saturation magnitude term. Severity never reads confidence - an abstention has no asserted threat class, so it is shown without a severity claim.
9. **Evidence bundle written.** The alert, its packet indices, feature vector, model hash and decision path are hashed into the chain.
10. **Analyst consumption.** The console reads the log through GET-only endpoints, or directly from a static snapshot with no server at all.

7. Features

7.1 Core detection features

Feature	What it does / why it exists	How it works	Output and dependencies
Visibility-matched training	Trains the detector under exactly the observability it will have at deployment. Exists because the measured cost of not doing so is 0.29 binary F1.	Each of the 103 corpus features is classified as forward-observable (35), backward-derived (33) or bidirectional-aggregate (35). Only the 35 forward-observable survive the tap.	A trained model whose feature set matches the tap. Depends on `ingest/replay.py`.
Streaming host-window aggregation	Recovers what the tap costs. This is the project's central mechanism.	16 per-host features over a 60-second tumbling window, computed in one pass with sketch-backed estimators.	WindowFeatures per (host, window). Depends on `features/sketches.py`.
Six detector heads	Covers the six threat classes the PS names, each with its own declared feature subset and known gaps.	Gradient-boosted trees per head, scored independently.	Per-class scores. Depends on the feature layer.
Conformal abstention	Lets the sensor decline rather than guess, with a coverage guarantee.	Split-conformal, Mondrian (per-class) mode, alpha = 0.1.	Calibrated decision or ABSTAIN. Depends on a held-out calibration split.
Hash-chained evidence	Makes each alert provable and retrospective edits detectable.	SHA-256 chain over alert records; each record links to its predecessor.	Append-only evidence log; a verify endpoint reports chain integrity.

Table 6. Core features, mechanisms and dependencies.

7.2 Encrypted-traffic features

Feature	What it does	Why it matters
JA4 client fingerprinting	Computes a JA4 fingerprint from the TLS ClientHello, with GREASE (RFC 8701) values stripped.	Measured on a real capture: JA3 produced 92 distinct fingerprints from 97 hellos while JA4 produced 7, because Chrome randomises extension order. JA3 is unusable as a rarity signal on modern browsers.

Feature	What it does	Why it matters
TLS ClientHello reassembly	Buffers and reassembles ClientHello messages spanning TCP segments.	94% of real ClientHellos in the project's own capture spanned segments; without reassembly the fingerprint feature had roughly 6% coverage.
QUIC Initial decryption	Derives Initial keys from the Destination Connection ID using the RFC 9001 published salt and decrypts to reach the ClientHello.	In the project's own capture UDP/443 beat TCP/443 by 80,291 to 32,069 packets. Most HTTPS is QUIC; without this the majority of encrypted web traffic is invisible to any TLS fingerprint.
DNS query-name analysis	Character entropy, n-gram (bigram) surprisal, label length, digit ratio; compression-pointer-safe qname parsing.	The forward-side replacement for NXDOMAIN rate, which is unobservable on a one-way tap.

Table 7. Encrypted-traffic and DNS feature set.

7.3 Operator-facing features

- **Alert list and detail.** Filterable by threat class and decision; the list omits the feature vector because it is too large, and the detail endpoint includes it.
- **Evidence verification.** A single endpoint recomputes the hash chain and reports any break, with the position of the first problem.
- **Detector-head transparency.** An endpoint returns each head's rationale and its declared known gaps, so an analyst can see what a head does not cover.
- **Severity that does not overstate.** Severity derives from kill-chain stage and volume magnitude, never from model confidence. Abstentions are displayed without a severity claim.
- **Offline operation.** Opening the HTML file with no server at all falls back to a static snapshot; the Findings tab and replay require the API.

8. Technical Implementation

Technology	Role	Why chosen	How used
Python 3.11+	Implementation language	Ecosystem fit for the numerical and ML stack; the PS is a research brief, not a latency-bound product.	All backend code; declared in `pyproject.toml` as requires-python >= 3.11.
NumPy / pandas	Numerical and tabular processing	Standard, vectorised, no alternative needed.	Feature matrices, corpus handling.
XGBoost	Gradient-boosted trees for the detector heads	Wins on tabular network features and remains interpretable via feature importance - important because explainability is graded.	One model per detector head.
scikit-learn	Metrics, splits, baseline models	Standard implementations of the evaluation primitives.	F1, AUC, calibration support.
SciPy	Statistical functions	Standard.	Distribution and statistical helpers.
cryptography	SHA-256 hashing for the evidence chain	Vetted primitive rather than a hand-rolled hash.	Evidence bundle chain links.
matplotlib	Figure generation	Produces the repository's result figures reproducibly.	`experiments/make_figures*.py`.
pytest	Test framework	Standard.	288 tests; the read-only constraint is enforced by one of them.

Technology	Role	Why chosen	How used
Plain HTML, CSS and JavaScript	Analyst console	The enclave has no internet route, so no CDN and no build step. A framework would mean the artefact in the enclave is not the artefact in the repository.	Single `index.html` plus a `data.js` snapshot.

Table 8. Technology choices with rationale.

8.1 Protocols and standards implemented

- **RFC 9001 (QUIC-TLS)** - Initial packet key derivation via HKDF-Expand-Label from the Destination Connection ID and the published version-1 salt; AES-128-ECB header protection; AES-128-GCM AEAD.
- **RFC 8701 (GREASE)** - GREASE cipher, extension and group values are stripped before fingerprinting, otherwise every hello looks unique.
- **JA3 / JA3S / JA4** - TLS client and server fingerprinting. JA4 sorts ciphers and extensions, which is what makes it stable against Chrome's extension-order randomisation.
- **DNS name compression** - qname parsing follows compression pointers with a backwards-only rule, so a malformed packet cannot induce a loop.
- **pcap and pcapng** - both container formats are decoded natively, including per-interface timestamp resolution in pcapng.

9. AI, ML and Data Science Components

9.1 Problem formulation

The learning task is **supervised multiclass classification over (host, window) cells and over flows**, evaluated primarily as binary attack-versus-benign F1 with macro-F1 reported alongside. Binary F1 is the headline because the temporal split leaves some attack classes present only in test - which is realistic for novel attack types but makes macro-F1 partly a measure of unseen-class generalisation.

9.2 Datasets

Dataset	Role	Scale and provenance
Corrected CIC-IDS2017 re-extraction	Primary real-traffic corpus for the visibility ablation	877,801 flows used; Zenodo record 22016274, CC-BY-4.0. Chosen over the original CSVs, which have documented label and feature-extractor defects.
Synthetic generator (in-repo)	Controlled instrument: lets every condition be varied in isolation	`ingest/synthetic.py`, 449 lines. Benign classes deliberately include periodic keepalives and telemetry so beacon detection is not trivial.
Live packet captures	Reality check on the packet path and the encrypted-traffic features	Two captures taken on an ordinary laptop by the team. Used for the JA3-versus-JA4, ClientHello-reassembly and QUIC findings, and for 279 distinct real benign domain names.
DGA family generators	Adversarial evaluation of the DNS head	Six families: uniform, conficker, necurs, banjori, suppobox, matsnu; 2,000 names per family.

Table 9. Datasets and their roles.

9.3 Two preprocessing decisions that determine whether the numbers mean anything

Source and destination IP are excluded from every feature set

The CIC-IDS2017 testbed uses fixed attacker addresses. A model given the IP addresses memorises them and reports a meaningless near-perfect score. The repository records this as the single most important preprocessing decision in the adapter.

Temporal split, never random

CIC-IDS2017 runs Monday to Friday with different attacks on different days. A random split places flows from the same attack episode on both sides of the split and inflates every score. The consequence of doing it correctly is that four classes (Botnet, Botnet-Attempted, DDoS, Portscan) are absent from training entirely, and 170,873 test flows belong to classes never seen in training.

9.4 Feature engineering

The 16 streaming host-window features are the engineered contribution. They are computed per (host, window) cell and include, by name as they appear in the code: `hw_src_n_flows`, `hw_src_n_peers`, `hw_src_n_dports`, `hw_src_fanout_per_flow`, `hw_src_fwd_pkts`, `hw_src_fwd_bytes`, `hw_src_iat_mean`, `hw_src_iat_cv`, `hw_src_peer_concentration`, `hw_dst_flows_per_src`, `hw_dst_fwd_pkts`, `hw_dst_fwd_bytes`, and TLS-related counters including `hw_src_tls_count` and `hw_src_tls_fp_novel`.

Feature-importance evidence that these are doing the work: in experiment 04, five of the top ten features by importance are host-window features, led by `hw_dst_flows_per_src` and `hw_src_fanout_per_flow`.

9.5 Model architecture and training

Gradient-boosted decision trees (XGBoost) per detector head. There is deliberately **no deep model in the detection path** and no LLM: the repository records latency, non-determinism and the evidence guarantee as the reasons. Tree ensembles also give per-feature importance directly, which is what the explainability requirement is graded on.

9.6 Inference pipeline

Raw packets or flows -> visibility filter -> flow assembly -> host-window aggregation -> per-head scoring -> conformal calibration -> severity -> evidence bundle. The pipeline is single-pass and streaming: peak traced memory in experiment 05 was 2.6632 MB while tracking 14,817 host baselines across 2,446 windows.

9.7 Evaluation metrics and why accuracy is not one of them

Accuracy is deliberately not a headline number - the classes are heavily imbalanced and accuracy hides that. The reported metrics are per-class precision, recall and F1; binary and macro F1; average precision; expected calibration error; abstention rate against accuracy-on-answered; and sustained packets per second with p99 latency.

9.8 Model limitations and failure cases

- **DGA is the weak class throughout** and is over-predicted. Against real benign names, dictionary-based families are close to unusable: suppobox combined AUC 0.6949, banjori 0.7475, against uniform-random 0.9787.
- **The generated corpus does not transfer.** A threshold calibrated on generated benign names flags 89.6% of real benign names. This is the strongest single piece of evidence that the generator is a controlled instrument and not a substitute for data.
- **Bigram surprisal needs a warm-up.** Below roughly 25 observed benign names the feature is near chance.
- **Host-window features alone fail at low attack density** - see Section 23.
- **The novelty channel is close to untested.** `hw_src_tls_fp_novel` fires on only 5 cells in the generated corpus.

10. Data Stores and Data Flow

There is no database

EKAGRA uses no relational or document database, no ORM and no external storage service. This is a deliberate consequence of the deployment target: a database is another service, another dependency and another attack surface inside an enclave that is supposed to have none. State is held in bounded in-memory structures; output is an append-only file.

10.1 Data stores that do exist

Store	Form	Contents	Lifecycle
Flow table	In-memory, LRU-evicted	Per-flow forward-only state keyed by 5-tuple	Entry created on first packet; closed by TCP flags, 15 s idle or 120 s active timeout; then discarded.
Host-window aggregates	In-memory, bounded by sketches	Per-(host, window) counters, cardinality and entropy estimators	Opened on first flow in the window; emitted after the window plus a 15 s lateness budget; then freed.
Per-host decayed baseline	In-memory, exponentially decayed	Long-running per-host volume baseline for the exfil head	Persists across windows; 14,817 entries measured on the full corpus at 2.66 MB peak traced memory.
Evidence log	Append-only JSONL file on disk	One record per alert: packet indices, feature vector, model hash, decision path, SHA-256 link to the previous record	Append-only. Never rewritten - rewriting is what the chain is designed to detect.
Console snapshot	Static `data.js`	A frozen export of a finished run for offline viewing	Regenerated when the demo is rebuilt.

Table 10. Data stores, their contents and lifecycles.

10.2 Data validation and security of the stored data

Validation is structural and happens at the boundary: malformed packets are rejected rather than guessed at, unparseable timestamps raise rather than defaulting, and DNS compression pointers may only point backwards. The evidence log is integrity-protected by the hash chain but is **not encrypted and not signed with an asymmetric key** - see Section 14 for what that does and does not guarantee.

11. APIs and Integrations

11.1 Internal API surface

Every endpoint is a GET. There are no write endpoints, and a backend test fails the build if one appears. Authentication is absent by design - see Section 14.

Endpoint	Purpose	Returns
`GET /api/health`	Liveness and log size	status, entries, log
`GET /api/summary`	Dashboard counts and provenance	counts, per-class breakdown, chain head, model hash
`GET /api/alerts`	Alert list with filtering and paging (limit, offset, threat_class, decision)	total, offset, limit, alerts[] - deliberately **without** the features field, which is too large for a list response
`GET /api/alerts/{seq}`	One alert in full	the alert record **including** its feature vector
`GET /api/verify`	Recompute and check the evidence hash chain	verified, problems[], entries, head
`GET /api/heads`	Detector-head transparency	the six heads with rationale and declared known gaps

Table 11. The complete read-only API surface.

11.2 External integrations

There are none, and that is the point

EKAGRA makes no outbound call to any third-party service: no threat intelligence, no reputation feed, no WHOIS, no cloud analytics, no CDN, no telemetry. This is enforced by a CI test rather than asserted in documentation. Consequently there are no API keys, tokens or credentials anywhere in the system - there is nothing to leak.

This is not merely a constraint the project tolerates. Experiment 01 measured that **removing an IP-reputation feed improved** macro-F1 from 0.926 to 0.971: at 55% recall and a 3% false-positive rate, a commercial-grade reputation feed is net negative once behavioural features are present. The enrichment rung cost was measured at -0.0448, that is, a benefit.

12. User Journey

The primary user is a security analyst working inside the enclave. They cannot query the outside world; the alert has to carry its own evidence.

1. The analyst opens the console - either against the local read-only API or by opening the HTML file directly with no server running.
2. The summary view shows counts by threat class, the current evidence-chain head, and the model hash that produced these alerts.
3. The analyst filters the alert list by threat class or by decision, including filtering specifically for ABSTAIN cases.
4. Selecting an alert opens the detail view, which loads the full feature vector for that alert.
5. The analyst reads the contributing features and the decision path to understand why the alert fired.
6. If the alert is an abstention, the console shows it without a severity claim, because an abstention asserts no threat class.
7. The analyst runs the verification endpoint to confirm the evidence chain is unbroken from the first record to the head.
8. The analyst consults the detector-head view to see what that head does not cover before drawing a conclusion.

13. User Interface and Experience

The console is a single `index.html` of roughly 37 KB with a companion `data.js` snapshot of roughly 503 KB. No framework, no bundler, no CDN, no build step.

Screen or component	Purpose	What the user sees and can do
Summary view	Orient the analyst quickly	Alert counts overall and per threat class; the evidence-chain head; the model hash. No actions - it is a status surface.
Alert list	Trie the queue down to what matters	Paged, filterable by threat class and decision. Severity is shown, except on abstentions. Clicking a row opens the detail view.
Alert detail	Answer 'why did this fire?'	The full feature vector, decision path, contributing packet indices and model hash for that single alert.
Evidence verification	Answer 'can I trust this log?'	A pass or fail on the hash chain, and the position of the first problem if there is one.
Detector heads view	Answer 'what does this not cover?'	Each head's rationale and its declared known gaps, including the explicitly handicapped ones.

Table 12. Console screens and their behaviour.

13.1 Design decisions that came out of defects

- **Abstentions no longer display a severity.** They previously rendered as HIGH. Severity derives from a threat class, and an abstention asserts none - so the value was meaningless. It is now hidden in the list and qualified in the detail view.
- **Severity is validated against a closed set before it reaches a CSS class attribute**, alongside HTML escaping - an injection-adjacent defect found and fixed during development.
- **A CSS specificity bug is pinned by a test.** A rule `.case small` outranked `.drv`, so a flex layout silently never applied; the selector is now scoped.

14. Security and Privacy

Stated plainly: there is no authentication and no authorisation

EKAGRA has no login, no user accounts, no role-based access control and no session management. This is a recorded scope decision, not an oversight - the repository lists it under what is deliberately not built. The security model assumes the console is reachable only from inside an already-restricted enclave. If it were exposed on a routable network, anyone reaching it could read every alert.

14.1 Security properties that are implemented and enforced

Property	Mechanism	Enforcement
No egress from the detection path	No outbound sockets anywhere downstream of ingest	**A CI test fails the build** if any component opens a socket. This is the strongest guarantee in the system.
No payload decryption	Only handshake metadata, sizes and timings reach feature code	Architectural: payload bytes are not passed into the feature layer.
Tamper-evident evidence	SHA-256 hash chain plus a keyed HMAC-SHA256 per bundle	A verify endpoint recomputes both. Seven tests attack it: edit, delete, reorder, feature-vector swap, whole-chain forgery, and survival across a JSONL round-trip.
Read-only API	GET-only endpoint surface	A backend test fails the build if a write endpoint appears.
Output-encoding safety in the console	Closed-set validation of severity plus HTML escaping	`tests/test_console_injection.py`.
Parser robustness	Compression pointers may only point backwards; malformed input is refused rather than guessed	Fuzz tests, including a fuzz test asserting JA4 computation never raises.

Table 13. Implemented security controls and how each is enforced.

14.2 Privacy posture

The sensor reads packet metadata, which includes DNS query names and IP addresses - inherently sensitive. Two mitigating facts are recorded: payloads are never decrypted, and the DGA experiment's results file explicitly notes that *no query name from the capture is stored* in the published results. Source and destination IP addresses are excluded from every model feature set, for correctness reasons, which incidentally means the trained models do not encode addresses.

14.3 Attack surface and residual threats

- **Unauthenticated console.** The largest residual risk. Mitigated only by network placement.
- **The evidence chain resists a whole-chain rewrite, but only while the key is held.** It is not a plain hash chain: each bundle also carries an **HMAC-SHA256** over its canonical form, keyed by a secret the enclave holds. An attacker who edits a record and recomputes every hash forward - which a competent attacker would do - produces a self-consistent chain, and the HMAC is what catches it. The residual risk is key compromise; there is still no external anchor and no asymmetric signature.
- **Adversarial evasion of the detectors is untested.** No experiment attempts to evade the heads by shaping traffic.
- **The tamper-evidence has been attacked, by seven automated tests.** They edit a value, delete an entry, reorder entries, swap a feature vector, forge the whole chain without the key, and confirm detection survives a JSONL round-trip. An API-level test covers the verify endpoint, and the console exposes a tamper control for demonstration.
- **Parser attack surface.** The project parses hostile input by definition. Fuzz tests exist for the wire parsers, which is the right control; coverage beyond those is not established.

15. Performance and Scalability

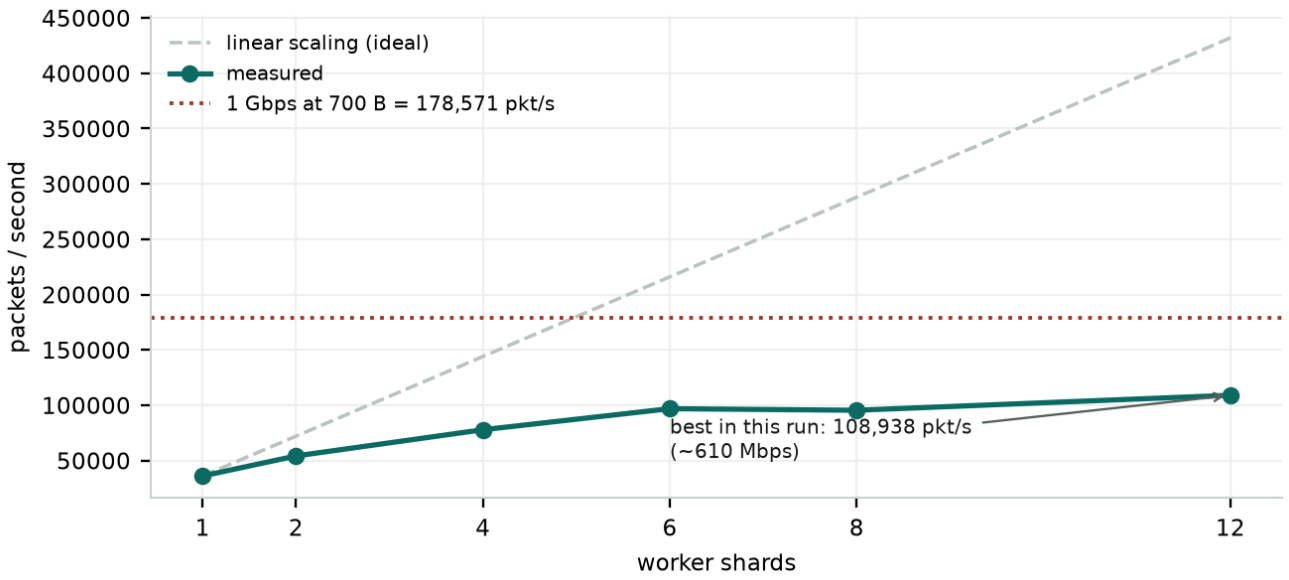

Figure 2. Sharded packet-pipeline scaling as recorded in exp07_verdicts.json. Scaling is strongly sublinear; the 1 Gbps target is not reached.

15.1 Measured performance

Measurement	Value	Source
Flow-level streaming throughput	171,501 flows/s	exp05_verdicts.json
Peak traced memory, full corpus	2.6632 MB for 14,817 host baselines	exp05_verdicts.json
Windows processed	2,446 over 877,801 flows	exp05_verdicts.json
Packet-pipeline throughput, single shard	35,992 packets/s	exp07_verdicts.json
Packet-pipeline throughput, 12 shards	108,938 packets/s (speed-up 3.03x, efficiency 0.25)	exp07_verdicts.json
1 Gbps requirement at 700-byte packets	178,571 packets/s	exp07_verdicts.json
End-to-end alert latency	75 s (60 s window + 15 s lateness budget)	exp06 / exp08 verdicts

Measurement	Value	Source
Assembler throughput	34,373 packets/s	exp06_verdicts.json

Table 14. Measured performance figures with their source files.

The throughput figure is contested between sources - see Section 26.6

The repository's RESULTS.md and README report a best case of 119,918 packets/s (about 672 Mbps) with a 2.70x speed-up. The current exp07_verdicts.json records a best case of 108,938 packets/s (about 610 Mbps) with a 3.03x speed-up. Both are real measurements from different runs; throughput benchmarks vary with machine load. The honest public statement is a range of roughly 610-672 Mbps, and in all versions the 1 Gbps target is not met.

15.2 Bottlenecks

Scaling efficiency falls from 1.00 at one shard to 0.25 at twelve. The repository attributes the gap to coordination rather than per-shard compute, and identifies kernel-level receive-side scaling or AF_PACKET fanout - so each shard reads its own NIC queue instead of being fed by a parent process - as the fix. Note also that sharding is not bit-exact: key agreement is essentially 1.0 but 8 to 21 cells per configuration show value mismatches, worst on `hw_src_iat_mean` and `hw_src_fwd_bytes`.

15.3 Streaming correctness under load

Experiment 05 confirms the streaming implementation matches an exact batch computation closely enough to be a substitute: binary F1 of 0.98120 batch against 0.98119 streaming for the host-window-only condition, with 14 of 16 features bitwise identical. Experiment 06 quantifies the lateness trade-off: a 0-second budget loses 10.17% of flows, 5 seconds loses 3.58%, and 15 seconds loses 0.0033% at the cost of 15 seconds of latency. That is why 15 seconds was chosen.

16. Deployment

16.1 Development environment

Python 3.11 or later with the dependencies declared in `ekagra/backend/pyproject.toml`: numpy, pandas, scikit-learn, xgboost, matplotlib, scipy, cryptography. The package is laid out under `src/` with pytest configured for `pythonpath = ["src"]` and `testpaths = ["tests"]`.

16.2 Running the system

```
cd ekagra/backend
python demo.py # starts the read-only API on 127.0.0.1:8000
# and serves ../frontend

# or, with no server at all:
# open ekagra/frontend/index.html directly
# (falls back to the data.js snapshot; Findings and replay need the API)

python analyse_pcap.py <capture.pcapng> # run the packet path on a capture
python -m pytest tests/ -q # 288 tests
python experiments/exp04_forward_features.py # the headline ablation
```

16.3 Production posture

- **No hosting, no cloud, no CI/CD pipeline to a cloud target.** The deployment target is an air-gapped enclave; shipping is copying the repository in.
- **No environment variables and no secrets.** There are no credentials because there are no external services.

- **No build step for the frontend.** Deliberate: a build step would mean the artefact running in the enclave is not the artefact in the repository.
- **The API binds to 127.0.0.1** in the demo entry point.
- [INFERENCE] There is no container image, service manifest, process supervisor or packaging step in the repository. Production deployment would require adding one; nothing in the code prevents it.

17. Project and Codebase Structure

Frontend and backend are separated at the top level so that the console can be worked on independently of the detection code.

```
ekagra/
ARCHITECTURE.md design rationale and the research argument
RESULTS.md full experiment log (926 lines)
backend/
pyproject.toml deps, pytest config
demo.py entry point: read-only API + static serve
analyse_pcap.py run the packet path over a capture file
src/ekagra/
ingest/ pcap.py, wire.py, quic.py, flow_assembler.py,
replay.py, synthetic.py, visibility.py,
diffuse.py, records.py
features/ streaming_host_window.py, host_window.py,
extractors.py, sketches.py, protocol.py
detect/ heads.py, base.py, severity.py
calibrate/ conformal.py
evidence/ bundle.py
pipeline/ sharded.py
api/ server.py
experiments/ exp01, exp03-exp11 + make_figures*
tests/ 16 test modules, 288 tests
results/ verdicts JSON, CSV, figures
frontend/
index.html the entire console (~37 KB)
data.js offline snapshot (~503 KB)
README.md API contract and the no-framework rationale
```

17.1 The most important files

File	Lines	Purpose and key responsibility
`ingest/pcap.py`	556	pcap and pcapng reader; IPv4/IPv6 and TCP/UDP decode; TLS ClientHello reassembly across TCP segments.
`features/streaming_host_window.py`	486	The central mechanism: single-pass per-host aggregation over tumbling windows with a lateness budget.
`ingest/wire.py`	484	DNS qname parsing (compression-pointer safe) and JA3 / JA3S / JA4 fingerprinting with GREASE stripping.
`ingest/synthetic.py`	449	Labelled traffic generator; benign classes include periodic keepalives so beaconing is not trivially separable.
`features/extractors.py`	405	Two deliberately separate classes: BidirectionalFeatures reproduces the published convention, UnidirectionalFeatures is the project's design.
`ingest/quic.py`	343	QUIC Initial key derivation and decryption per RFC 9001; CRYPTO frame reassembly.
`ingest/replay.py`	302	Corpus adapters and the 103-feature forward/backward/bidirectional classification with a printable column audit.
`evidence/bundle.py`	283	Hash-chained evidence records.
`pipeline/sharded.py`	280	Multi-process sharding by host.
`features/protocol.py`	275	DNS and TLS protocol feature extraction.

File	Lines	Purpose and key responsibility
`api/server.py`	251	The GET-only HTTP API.
`calibrate/conformal.py`	247	Split-conformal calibration, Mondrian mode, ABSTAIN decision.
`detect/severity.py`	201	Kill-chain stage impact and volume-saturation magnitude. Never reads confidence.
`detect/heads.py`	191	The six detector heads with declared feature subsets and known gaps.

Table 15. Principal source files by size, with responsibilities.

18. Algorithms and Core Logic

18.1 Streaming cardinality and entropy (sketches)

Problem. Per-host distinct-peer and distinct-port counts, and source-IP entropy, must be computed at line rate over unbounded input. **Why not a dictionary.** Memory grows with cardinality, which an attacker controls. **Approach.** HyperLogLog for cardinality, Count-Min Sketch for frequency, Misra-Gries for heavy hitters, and exponentially-decayed counters for baselines. **Refinement that matters.** Estimators start *exact* and promote to sketches only above 256 distinct values, which is why 14 of 16 features stay bitwise identical to an exact batch computation. **Complexity.** O(1) memory per host per estimator, O(1) update per flow.

18.2 Tumbling windows with a lateness budget

Problem. Flows arrive out of order; closing a window immediately discards late arrivals, closing it late costs alert latency. **Approach.** A 60-second tumbling window plus a configurable lateness budget; the window emits at window-end plus budget. **Why 15 seconds.** Measured: 0 s loses 10.17% of flows, 5 s loses 3.58%, 15 s loses 0.0033%. 15 seconds buys a 3,000-fold reduction in loss for 15 seconds of latency.

18.3 Split-conformal prediction with Mondrian conditioning

Problem. Turn an uncalibrated score into a decision with a guarantee, and allow refusal. **Approach.** Compute non-conformity scores on a held-out calibration set; take the (1 - alpha) quantile; at inference, emit a prediction set and abstain when the set is not a singleton. **Mondrian** means the quantile is taken per class rather than marginally, so a rare class cannot have its coverage silently absorbed by a common one. **Measured at alpha = 0.1:** empirical coverage 0.9080, abstention rate 0.0911, accuracy on answered 0.9955, expected calibration error 0.00084. **Honest caveat:** per-class coverage is not uniform - one class sits at 0.5, which the results file records.

18.4 The 103-feature visibility classification

Problem. Decide, per corpus feature, whether a one-way tap can compute it. **Approach.** Explicit three-way classification: 35 forward-observable (kept), 33 backward-derived (dropped by definition), 35 bidirectional-aggregate (dropped). **The subtle case.** A feature like `Flow IAT Mean` is not "forward IAT with noise" - on a one-way tap the sensor computes a *different quantity* over forward packets only, which the CSV does not contain. It is dropped rather than assumed observable. A `column_audit()` function prints the full classification so a reviewer can dispute any single assignment.

18.5 JA4 fingerprinting and GREASE stripping

Problem. Identify a TLS client from the handshake without decrypting anything. **Why JA3 fails.** JA3 hashes the cipher and extension lists in wire order; Chrome has randomised extension order since version 110, so the same client produces a different fingerprint each connection - measured as 92 distinct fingerprints from 97 hellos. **Approach.** JA4 sorts ciphers and extensions before hashing, giving 7 fingerprints over the same traffic. GREASE values (RFC 8701) are detected by the bit pattern `(v and 0x0F0F) == 0x0A0A` with matching high and low bytes, and stripped first.

18.6 QUIC Initial decryption

Problem. Most HTTPS is now QUIC, whose handshake is encrypted - but the ClientHello must be readable by a server that has no prior state. **Approach.** RFC 9001 specifies that Initial packet keys derive from the Destination Connection ID, which is in the clear, using a published salt. The implementation performs HKDF-Expand-Label, removes AES-128-ECB header protection, decrypts the AES-128-GCM AEAD, and reassembles CRYPTO frames to recover the ClientHello. **Boundary:** this reads a handshake that is designed to be readable. It is not payload decryption - 1-RTT application keys remain out of reach.

18.7 Severity assignment

Problem. Rank alerts without letting model confidence masquerade as impact. **Approach.** Severity combines a kill-chain stage impact with a volume-saturation magnitude term, using volume only where volume *is* the impact (exfiltration, encrypted transfer) and reading the relevant side of the cell in a side-agnostic way. **Deliberate exclusion:** severity never reads confidence, which is why an abstention carries no severity. **Defects this design fixed:** medians of 0.00 for scan and beacon across 90,794 alerts, and a present-but-zero value being conflated with an absent one.

19. Innovation and Differentiation

- **The contribution is a measured profile, not an architecture.** Most submissions to a detection problem propose a model. This one measures what the deployment can observe first, then builds to that. The procedure transfers to any sensor placement.
- **The diagnosis is counter-intuitive and was verified by control.** The naive reading is that a one-way tap fails because a direction is lost. The measurement says the recoverable loss is per-host state - and the control that established this also narrowed the project's own headline claim.
- **Abstention as a first-class output.** A calibrated ABSTAIN with a coverage guarantee is materially different from a low-confidence prediction, and it is the honest behaviour under a constraint that genuinely removes evidence.
- **Evidence that survives the enclave.** In an air-gapped environment an analyst cannot re-query anything, so an alert must carry its own proof. Hash-chained bundles with packet indices and a model hash are an unusual thing to build for a hackathon and are directly responsive to the deployment.
- **The read-only guarantee is enforced by a test, not a promise.** A reviewer can delete the test and watch the build stop failing; that is a stronger claim than any paragraph.
- **Encrypted-traffic realism.** JA3 was implemented, measured against the team's own traffic, found unusable, and replaced with JA4 - and QUIC, initially declared a hard ceiling, was subsequently reached. Both are recorded as corrections.

Claims deliberately not made

No claim of being first, best, or state of the art is made anywhere in this document or in the repository. The comparison in Section 20 is restricted to properties that are architecturally determined or measured.

20. Existing Solutions Compared With EKAGRA

Verified facts are marked [V]; qualitative analysis is marked [A].

Dimension	Signature IDS (Suricata / Zeek)	Commercial NDR (Darktrace / Vectra class)	EKAGRA
Works behind a one-way tap	[A] Degraded - many rules need the response side	[A] No - requires a cloud callback	[V] Yes, by construction
Licence cost	[A] Free and open source	[V] Published at roughly USD 8-22 per protected IP per year	[V] Zero

Dimension	Signature IDS (Suricata / Zeek)	Commercial NDR (Darktrace / Vectra class)	EKAGRA
Requires an egress path	[A] Yes, for rule updates	[A] Yes, for analytics and licensing	[V] No - enforced by a CI test
Data leaves the enclave	[A] Rule feed only	[A] Yes - telemetry to vendor cloud	[V] No
Detection quality on the tap	[A] Not measured by this project	[A] Not measured by this project	[V] 0.9810 binary F1 on 877,801 real flows
Throughput	[A] Well beyond this project's	[A] Appliance-class	[V] ~610-672 Mbps measured; 1 Gbps not reached
Abstention with a guarantee	[A] No	[A] Not exposed as such	[V] Yes - conformal, alpha = 0.1, coverage 0.908
Tamper-evident per-alert evidence	[A] Logging only	[A] Vendor-dependent	[V] SHA-256 chain with packet indices and model hash
Maturity and operational support	[A] Very high - years of production use	[A] High - commercial support	[A] **Prototype.** No hardening, no packaging, no support
Threat-class coverage breadth	[A] Very broad rule libraries	[A] Broad	[V] Six classes, of which DGA is measurably weak

Table 16. Comparison across dimensions. EKAGRA is not superior across the board - it is superior on the axes the deployment forces, and clearly behind on maturity and breadth.

21. Use Cases

21.1 Primary use case - monitoring a diode-protected enclave

Element	Detail
User	Security analyst inside an NTRO or NCIIPC-designated enclave
Situation	Traffic crosses a data diode into the enclave; nothing may be sent back out
Problem	The commercial or open-source detector they would otherwise deploy assumes bidirectional visibility and a reachable threat-intelligence service
How EKAGRA is used	Deployed on the enclave side of the diode; consumes the mirrored stream; analyst works the console locally
Expected result	Detection quality comparable to a full-visibility deployment (0.9810 against 0.9793 measured), with explicit abstention where evidence is genuinely insufficient
Benefit	No licence, no egress path, no telemetry leaving the enclave, and an audit trail per alert

Table 17. Primary use case.

21.2 Secondary use cases

- **Asymmetric SPAN monitoring.** A campus or ISP mirror port that carries only one direction - common with asymmetric routing. Same visibility profile, same sensor.
- **Post-incident capture analysis.** `analyse_pcap.py` runs the full packet path over a stored capture, producing the same alerts and evidence bundles offline.
- **Methodology reuse.** An organisation deploying any sensor in a constrained position can run the visibility-classification procedure to establish which of its features survive before training.
- **Teaching and evaluation.** The repository is an unusually complete worked example of pre-registered experimental discipline applied to a security ML problem. [INFERENCE: this use is not stated in the repository; it follows from its structure.]

22. Advantages

Category	Advantage
Technical	Detection quality on a one-way tap is level with full visibility on the tested corpus (0.9810 against 0.9793). Streaming implementation matches batch to within 0.00001 binary F1 with 14 of 16 features bitwise identical. Bounded memory: 2.66 MB for 14,817 hosts.
Operational	Runs air-gapped with no external dependency. Abstains rather than guessing, at a measured 9.11% rate with 99.55% accuracy on answered cases. Every alert is independently verifiable through the hash chain.
Economic	Zero licence cost against a per-IP commercial model, and the reputation-feed dependency it removes was measured as net negative anyway (macro-F1 0.926 to 0.971 without it).
Security and compliance	No egress path to be abused, no third-party telemetry, no credentials to leak, no payload decryption. Sovereign by construction.
Scalability	Sharding by host is implemented and demonstrated (3.03x on 12 workers); the remaining gap is diagnosed as coordination, with a named fix.
Evaluative credibility	42 pre-registered predictions with 17 refuted, verdicts printed by the runs themselves. 288 tests. One-command reproduction.

Table 18. Advantages by category.

23. Limitations and Current Gaps

This section is deliberately unflattering

The repository's own discipline is to publish the control that narrows each claim. The following are real, current, and stated without mitigation language.

23.1 Capability limitations

- **The headline was narrowed by the project's own control.** 16 host-window features *alone* score 0.9812 - marginally above the full sensor. On this corpus the visibility question largely dissolves, so the claim is about host state, not about direction.
- **And that narrowing itself does not generalise.** At low attack density, where attack flows share a cell with benign traffic at ratios from 59:1 to 188:1, host-window-alone falls from 0.599 to 0.28 macro-F1 while full visibility holds 0.77-0.86. The rule is: keep per-host state, but never run on it alone.
- **1 Gbps is not reached.** Roughly 610-672 Mbps depending on run, against a 178,571 packet/s requirement.
- **DGA is measurably weak.** Dictionary-based families are close to unusable: suppobox 0.6949 and banjori 0.7475 combined AUC. At a 1% false-positive rate, conficker recall is 0.0 and suppobox 0.004.
- **The generated corpus does not transfer.** A threshold calibrated on generated benign names flags 89.6% of real benign names.
- **Sharding is not bit-exact.** 8 to 21 cells per configuration disagree with the single-process result.
- **Packet-size and timing sequences are missing**, and the PS names them beside fingerprints. The code states they do not survive flow-level aggregation - but a full packet parser now exists, so the stated reason no longer holds. This is the clearest gap against the PS text.
- **The TLS-novelty channel is close to untested** - it fires on 5 cells in the generated corpus.

23.2 Engineering and product gaps

- No authentication, authorisation or multi-tenancy of any kind.
- No packaging, container image, service manifest or process supervisor.

- No adversarial-evasion testing.
- The packet path (experiments 06-09) is validated on generated packets only; CIC-IDS2017 ships flow records and cannot exercise an assembler.
- The synthetic generator does not emit the final ACK after a FIN, so the project's own corpus cannot exercise the TCP teardown path.

23.3 Documentation gaps

`ARCHITECTURE.md` refers to modules that do not exist under those names - `ingest/diode.py` and `features/flow_state.py`. The actual files are `ingest/visibility.py`, `ingest/flow_assembler.py` and `features/streaming_host_window.py`. See Section 26.6.

24. Future Scope

Prioritised by impact multiplied by feasibility, highest first within each horizon.

24.1 Short term

Item	Why it matters	Feasibility
Packet-size and inter-arrival sequence features	Closes the clearest remaining gap against the PS text. The stated blocker - that sequences do not survive flow aggregation - no longer applies now that a per-packet parser exists.	High - the parser yields per-packet records already
Anchor the evidence chain head outside the enclave	The chain plus HMAC resists forgery while the key is held; it cannot survive key compromise. Publishing the head periodically to an append-only ledger would close that - though it reintroduces an egress path and would need to be one-way.	Medium - design work, not just code
Re-measure throughput end to end and publish a single number	Resolves the 610 against 672 Mbps discrepancy and covers parsing and reassembly rather than feature extraction alone.	High
Test the TLS-novelty channel on real fingerprints	The channel is currently near-untested; real JA4 fingerprints from the team's own captures exist.	High
Reconcile ARCHITECTURE.md with the code	Stale module names undermine a reviewer's confidence in everything else in the document.	High

Table 19. Short-term roadmap.

24.2 Medium term

- **Kernel-level packet fanout** (receive-side scaling or AF_PACKET) so each shard reads its own NIC queue - the named fix for the coordination bottleneck, and the route to line rate.
- **Real-corpus validation of the packet path.** Experiments 06-09 run on generated packets; a labelled packet-level corpus would let the assembler and the protocol parsers be validated on real traffic.
- **Authentication and authorisation for the console**, if it is ever to sit anywhere other than an already-restricted network.
- **Improve the DGA head on dictionary-based families**, where word-list and pronounceability features are the obvious direction.
- **Packaging** - a container image or an installable artefact with a documented service definition.

24.3 Long term

- **Line-rate operation at 10 Gbps**, which is an engineering programme rather than a feature: kernel bypass, per-queue affinity and a rewrite of the hot path in a compiled language.

- **Anchoring the evidence chain externally**, which is the one thing the current design cannot do for itself: the chain plus HMAC defeats forgery while the key is held, but not key compromise. Given the theme is Blockchain and Cybersecurity, periodically publishing the chain head to an append-only ledger outside the enclave is the natural direction - though it reintroduces an egress path and would need to be one-way.
- **Formal deployment trial** with a real diode operator, which is the only way to establish operational value.
- **Generalising the visibility-profile procedure into a reusable tool** that, given a corpus and a sensor placement, emits the feature classification and the matched training set automatically.

25. Testing and Validation

25.1 Test suite

288 tests across 16 modules, collected and counted directly from the repository during authoring. The suite is not incidental to the argument - several tests exist specifically to enforce claims made in the documentation.

Test module	What it protects
`test_readonly_constraint.py`	**Fails the build if anything in the detection path opens a socket.** This test *is* the read-only guarantee.
`test_pcap.py`	pcap and pcapng decoding, ClientHello reassembly across segments.
`test_wire.py`	DNS qname parsing, JA3/JA3S/JA4, GREASE handling; includes a fuzz test asserting JA4 computation never raises.
`test_quic.py`	QUIC Initial key derivation and decryption.
`test_streaming_host_window.py`	Window boundaries, lateness handling, sketch promotion thresholds.
`test_flow_assembler.py`	Flow closure by flags, idle timeout and active timeout.
`test_cicids_adapter.py`	Pins the forward/backward/bidirectional feature classification invariants.
`test_evidence.py`	Hash-chain construction and verification.
`test_severity.py`	Severity never reads confidence; present-but-zero is distinguished from absent.
`test_console_injection.py`	Closed-set severity validation and HTML escaping.
`test_api.py`	Endpoint contract; fails if a write endpoint appears.
`test_sharded_pipeline.py`	Sharded and single-process agreement.
`test_reproducibility.py`	Deterministic seeds and reproducible outputs.
`test_detector_heads.py`, `test_diffuse.py`	Head behaviour and the low-density re-timing transform.

Table 20. Test modules and the properties they enforce.

25.2 Experimental validation methodology

Each experiment file carries **pre-registered predictions written before the first run**. The run prints a verdict for each and persists it to `results/exp*_verdicts.json`. Predictions are never edited after a run; where one was badly worded it is annotated, not reworded. The tally for EKAGRA is **42 predictions, 17 refuted**, computed directly from the verdict files during authoring.

Experiment	Question	Predictions	Refuted
exp01	Synthetic visibility ablation	4	4
exp03	CIC-IDS2017 visibility ablation	3	1
exp04	Do host-window features close the gap?	3	0
exp05	Does streaming match batch?	4	0

Experiment	Question	Predictions	Refuted
exp06	Packet pipeline and lateness budget	4	1
exp07	Sharded throughput	5	4
exp08	Conformal calibration and evidence	3	0
exp09	Per-head detector quality	4	2
exp10	Low-rate diffuse attacks	6	2
exp11	DGA against real benign names	6	3
Total		**42**	**17**

Table 21. Pre-registered predictions per experiment, from the verdict files.

26. Results and Evidence

26.1 The headline ablation

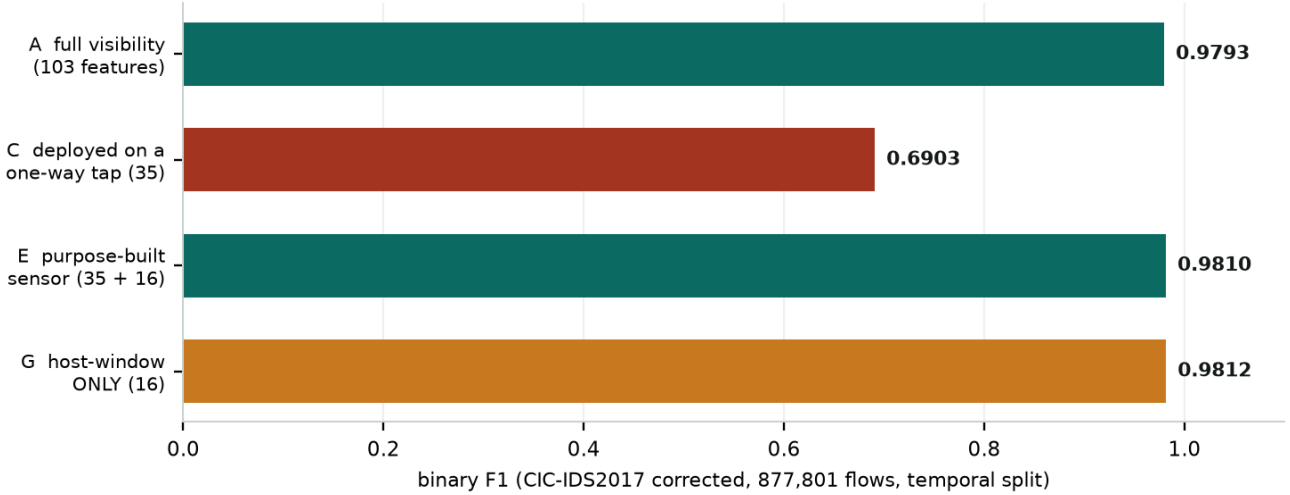

Figure 3. The visibility ablation on 877,801 real flows. The orange bar is the project's own control: host-window features alone match the full sensor, which narrowed the original claim.

Condition	Features	Binary F1	Macro F1
A - full visibility	103	0.9793	0.3999
C - deployed on a one-way tap	35 forward-observable	0.6903	0.3110
E - purpose-built sensor	35 + 16 host-window	0.9810	0.4660
F - full visibility plus host-window	103 + 16	0.9811	0.5593
G - host-window ONLY (control)	16	**0.9812**	**0.5995**

Table 22. Experiment 04, from exp04_verdicts.json. Window size 60 s. Note that macro-F1 tells a different story from binary F1 - the temporal split leaves four classes absent from training entirely.

26.2 Other measured results

Result	Value	Experiment
Streaming matches batch	0.98120 batch vs 0.98119 streaming; 14 of 16 features bitwise identical	exp05
Peak memory	2.6632 MB for 14,817 host baselines	exp05
Flow throughput	171,501 flows/s	exp05

Result	Value	Experiment
Lateness budget	0 s loses 10.17%; 5 s loses 3.58%; 15 s loses 0.0033%	exp06
Conformal calibration	coverage 0.9080, abstention 0.0911, accuracy on answered 0.9955, ECE 0.00084	exp08
Threat-class co-occurrence	0.082% of test cells	exp09
Removing the reputation feed	macro-F1 0.926 to 0.971 (enrichment rung cost -0.0448)	exp01
Low-density stress test	one-way tap holds (0.937-0.994 binary F1); host-window-alone collapses 0.599 to 0.28 macro-F1	exp10
DGA against real benign names	uniform 0.9787; necurs 0.9434; conficker 0.9402; matsnu 0.8941; banjori 0.7475; suppobox 0.6949 combined AUC	exp11
Generated-to-real threshold transfer	flags 89.6% of 279 real benign names	exp11

Table 23. Principal measured results with their source experiments.

26.3 Findings that came from real captured traffic

- **94% of real TLS ClientHellos span TCP segments** - 91 of 97 failed before a reassembler was built.
- **JA3 gave 92 distinct fingerprints where JA4 gave 7** over the same 97 hellos, because Chrome randomises extension order.
- **UDP/443 beat TCP/443 by 80,291 to 32,069 packets** - most HTTPS in the capture was QUIC.

26.4 Measured versus projected

Everything in the two tables above is **measured** and reproducible from the repository. The only **projected** figure used anywhere in this project's materials is the Rs 13-37 lakh per year licence-avoidance estimate, which is arithmetic from published per-IP list pricing applied to an assumed 2,000-IP enclave.

26.5 Contradictions found between sources, and the ruling on each

The instruction for this report was to cross-reference sources rather than trust one. Five contradictions were found. In each case the **results JSON files are treated as authoritative**, because they are written by the runs themselves, with prose documents second and slides third.

#	Contradiction	Ruling
D1	Throughput: RESULTS.md and README report 119,918 packets/s (672 Mbps, 2.70x). exp07_verdicts.json records 108,938 packets/s (610 Mbps, 3.03x).	**Both are genuine runs**; throughput varies with machine load. Report a range of 610-672 Mbps. Recommendation: re-run and publish one number. Materially affects the slides, which say 672 Mbps.
D2	ARCHITECTURE.md section 5 names `ingest/diode.py` and `features/flow_state.py`; neither exists.	**The code is authoritative.** The files are `ingest/visibility.py`, `ingest/flow_assembler.py` and `features/streaming_host_window.py`. Documentation drift; harmless to results, corrosive to reviewer confidence.
D3	Two different experiments are both described as 'the ablation' with different numbers: exp03 gives 0.9877 / 0.5222 / 0.6833, exp04 gives 0.9793 / 0.6903 / 0.9810 / 0.9812.	**Both are correct and are different experiments.** exp03 is the first CIC-IDS2017 ablation; exp04 adds the host-window condition. The slides and this report quote exp04. Always name the experiment when quoting.
D4	A memory note recorded DGA figures of 0.9925 uniform AUC, banjori 0.6006 and a 94.4% transfer rate.	**All three were stale and have been corrected.** exp11_verdicts.json gives uniform 0.9787, banjori 0.7475 and 89.6%. The repository's own RESULTS.md and JUDGE_QA.md were already correct.
D5	ARCHITECTURE.md section 2 presents the host-window finding without the low-density caveat in its opening statement.	**Resolved within the same document**, which goes on to state the exp10 result. Readers should take the narrowed form: keep per-host state, never run on it alone.

Table 24. Contradictions between repository sources and the ruling on each.

27. Risks and Challenges

Risk	Type	Mitigation
Throughput does not meet a 1 Gbps link	Technical	Measured and published rather than hidden. Named fix: kernel-level fanout so shards read their own NIC queue. Interim: deploy per-segment rather than per-link.
The corpus is a single testbed capture from 2017	Data	Acknowledged. The corrected re-extraction was chosen over the defective original, IP addresses are excluded, and the split is temporal. Additional corpora remain future work.
The generated corpus does not represent real traffic	Data	Measured directly (89.6% transfer failure) and reported as the strongest evidence that the generator is an instrument, not a substitute.
Unauthenticated console exposed beyond the enclave	Security	Currently mitigated only by network placement. This is a genuine open risk and is stated as one.
Evidence log rewritten wholesale by an attacker with file access	Security	The chain detects edits, not a full consistent rewrite. External anchoring is listed as long-term future work.
Adversarial evasion of the detector heads	Technical	Untested. No mitigation currently claimed.
Operator adoption without support or packaging	Adoption	Prototype status is stated plainly. Packaging is medium-term roadmap.
Reviewer finds a stale figure across documents	Operational	Section 26.6 catalogues every contradiction found, with rulings, rather than leaving them to be discovered.

Table 25. Risks with mitigations.

28. Presentation Knowledge Base

This section is written for someone building a deck from scratch. For each recommended slide: objective, exact points, the technical detail that earns credit, a suggested visual, what to say, and what to keep off.

Scoring context. SIH evaluators spend two to three minutes per submission. The official weighting used for this project's own slides is: innovation and uniqueness 25%, problem understanding and clarity 20%, technical feasibility 20%, impact and scalability 20%, presentation quality 15%. The official template caps the idea submission at six slides including the title and requires PDF.

28.1 Slide 1: Title

Element	Content
Objective	Identify the submission unambiguously.
Points on the slide	PS ID SIH26145; the PS title verbatim; organisation NTRO; theme Blockchain and Cybersecurity; category Software; team AlgoRhythms; team ID.
Technical detail that earns credit	Nothing technical.
Suggested visual	Clean typographic hierarchy only.
What the presenter says	"Problem statement SIH26145 from NTRO - detecting cyber threats in unidirectional IP traffic."
Keep off this slide	Do not put the abstract here.

28.2 Slide 2: Problem

Element	Content
Objective	Make the constraint vivid in fifteen seconds.
Points on the slide	A data diode carries traffic in and provides no path back. Published detectors assume both directions plus a reachable threat-intel service. Measured cost of that assumption: 0.9793 to 0.6903 binary F1.
Technical detail that earns credit	The three PS constraints: read-only ingest, no payload decryption, streaming not batch.
Suggested visual	A simple diode graphic with a crossed-out return arrow.
What the presenter says	"The enclave sees everything and can say nothing back. That is the whole problem."
Keep off this slide	Do not list all six threat classes here - it dilutes the single idea.

28.3 Slide 3: Motivation

Element	Content
Objective	Establish that this is current and consequential.
Points on the slide	CERT-In handled 29.44 lakh incidents in 2025, up 85% since 2023. Unidirectional gateways are becoming standard for OT and defence segments. Sovereignty requirement for CII.
Technical detail that earns credit	Name NCIIPC and CERT-In explicitly.
Suggested visual	One stat block, not a paragraph.
What the presenter says	"The control that protects the enclave is what blinds the monitoring inside it."
Keep off this slide	Do not pad with generic cybercrime statistics.

28.4 Slide 4: Existing System

Element	Content
Objective	Win the 25% innovation criterion. **Highest-value slide after the idea.**

Element	Content
Points on the slide	A four-row grid: Suricata/Zeek (rule-update path is an egress path); Darktrace/Vectra-class NDR (cloud callback, USD 8-22 per IP per year); Owl/Waterfall diodes (move data, do not detect); EKAGRA (built for the constraint).
Technical detail that earns credit	The distinction that each fails for a *different* reason.
Suggested visual	A comparison table - scannable in ten seconds.
What the presenter says	"Three categories of existing solution, three different reasons none of them can be deployed here."
Keep off this slide	Do not write this as prose. A grid or nothing.

28.5 Slide 5: Proposed Solution

Element	Content
Objective	State the idea and the evidence together.
Points on the slide	A read-only sensor that keeps 60-second per-host state. Two-bar chart: 0.6903 conventional against 0.9810 EKAGRA. Stat cards: 0.9810, zero licence, 610-672 Mbps, 17 of 42 predictions refuted.
Technical detail that earns credit	877,801 real flows, temporal split, IPs excluded from features.
Suggested visual	Native bar chart with the failing bar in a warning colour.
What the presenter says	"Same traffic, same split. The gap is not the lost direction - it is lost per-host state, and that is recoverable."
Keep off this slide	Do not put the full architecture here.

28.6 Slide 6: Innovation

Element	Content
Objective	Say precisely what is new.
Points on the slide	A measured visibility profile rather than a proposed architecture; the counter-intuitive diagnosis verified by control; abstention as a first-class output; evidence that survives an enclave; the read-only guarantee enforced by a test.
Technical detail that earns credit	That the control narrowed the team's own headline.
Suggested visual	Icon row, one per point.
What the presenter says	"We did not start by proposing a model. We started by measuring what the tap can see."
Keep off this slide	Never claim first, best, or state of the art.

28.7 Slide 7: Architecture

Element	Content
Objective	Prove technical depth in one image.
Points on the slide	The eight-stage pipeline with the enforced one-way banner.
Technical detail that earns credit	Sketch types by name (HyperLogLog, Count-Min, Misra-Gries); 60 s window plus 15 s lateness; Mondrian conformal; SHA-256 chain.
Suggested visual	The architecture diagram from this report (Figure 1).
What the presenter says	"Nothing downstream of ingest can call anything upstream, and a CI test fails the build if it tries."
Keep off this slide	Do not label every arrow. Let the diagram breathe.

28.8 Slide 8: Workflow

Element	Content
Objective	Show the lifecycle of one alert.
Points on the slide	Packet arrives, tap model applied, flow assembled, host-window aggregated, window closes at +75 s, six heads score, conformal calibrates or abstains, severity assigned, evidence chained, analyst reads it.
Technical detail that earns credit	The 75-second latency figure and why 15 s of it is the lateness budget.
Suggested visual	A numbered vertical flow.
What the presenter says	"Seventy-five seconds from packet to provable alert."
Keep off this slide	Do not repeat the architecture diagram.

28.9 Slide 9: Technologies

Element	Content
Objective	Answer the stack question before it is asked.
Points on the slide	Python 3.11+, XGBoost, NumPy/pandas, scikit-learn, SciPy, cryptography, pytest. Frontend: plain HTML/CSS/JS with no framework, no CDN, no build.
Technical detail that earns credit	Why no framework: a build step means the artefact in the enclave is not the artefact in the repository.
Suggested visual	A compact stack table.
What the presenter says	"No LLM anywhere in the detection path - latency, non-determinism, and it would break the evidence guarantee."
Keep off this slide	Do not list every transitive dependency.

28.10 Slide 10: AI / ML

Element	Content
Objective	Show the modelling is deliberate.
Points on the slide	Supervised multiclass over (host, window) cells; GBDT per head; 16 engineered host-window features; conformal calibration at alpha = 0.1.
Technical detail that earns credit	The two preprocessing decisions: IPs excluded, temporal split - and that four classes are consequently absent from training.
Suggested visual	Feature-importance bar showing five host-window features in the top ten.
What the presenter says	"Accuracy is not our headline because these classes are heavily imbalanced and accuracy hides that."
Keep off this slide	Do not show a confusion matrix at this size.

28.11 Slide 11: Features

Element	Content
Objective	Inventory without drowning the reader.
Points on the slide	Six detector heads; streaming host-window aggregation; conformal abstention; hash-chained evidence; JA4 and QUIC handling; the read-only console.
Technical detail that earns credit	Name the six threat classes exactly as the PS does.
Suggested visual	Two-column icon list.
What the presenter says	"Six heads, each declaring its own known gaps - including the ones we have not solved."
Keep off this slide	Do not describe every feature's implementation.

28.12 Slide 12: Demo Flow

Element	Content
Objective	Set expectations for the live demo.
Points on the slide	The five console steps from Section 31.
Technical detail that earns credit	That the console runs with no server and no network.
Suggested visual	Three screenshots: summary, alert detail, verification result.
What the presenter says	"This is running with the network cable out."
Keep off this slide	Do not narrate a demo you will then also perform.

28.13 Slide 13: Results

Element	Content
Objective	Evidence, including the uncomfortable part.
Points on the slide	The four-bar ablation with the control highlighted; 288 tests; 17 of 42 predictions refuted; conformal coverage 0.908 at 9.11% abstention.
Technical detail that earns credit	That the control at 0.9812 narrowed the team's own claim.
Suggested visual	The ablation chart (Figure 3 of this report).
What the presenter says	"The tallest bar is our own control, and it is the reason we narrowed our claim instead of defending it."
Keep off this slide	Do not hide the control. A judge who finds it alone has caught you.

28.14 Slide 14: Advantages

Element	Content
Objective	Translate technical results into operator value.
Points on the slide	Zero licence and zero egress; abstains rather than guessing; every alert provable; sovereign by construction; bounded memory.
Technical detail that earns credit	The reputation-feed measurement: removing it improved macro-F1 from 0.926 to 0.971.
Suggested visual	Four cards.
What the presenter says	"Removing the threat-intel dependency made detection better, not worse. The constraint costs nothing here."
Keep off this slide	Do not repeat the ablation numbers again.

28.15 Slide 15: Future Scope

Element	Content
Objective	Show the roadmap is real and prioritised.
Points on the slide	Short: packet-size and timing sequences, adversarial evidence test, throughput re-measurement. Medium: kernel fanout, real-corpus packet validation, authentication. Long: line rate at 10 Gbps, external anchoring of the evidence chain.
Technical detail that earns credit	That packet-size sequences are a named PS requirement not yet met.
Suggested visual	A three-horizon timeline.
What the presenter says	"The nearest gap is one the PS names and we have not closed - and we now have the parser that makes it possible."
Keep off this slide	Do not promise anything not on this list.

28.16 Slide 16: Conclusion

Element	Content
Objective	Leave one sentence behind.
Points on the slide	Measured the visibility profile; built the sensor to it; recovered 0.6903 to 0.9810; published the control that narrowed the claim.
Technical detail that earns credit	42 predictions, 17 refuted, verdicts printed by the runs.
Suggested visual	A single line of text.
What the presenter says	"We can hand you the repository and one command reproduces every number on these slides, including the ones that went against us."
Keep off this slide	Do not add new information in the conclusion.

29. Pitch Material

29.1 30-second pitch

"Secure enclaves are monitored through a data diode - the sensor sees the traffic but has no path back. Every published intrusion detector assumes it can see both directions and call a threat-intel service. We measured what that costs on 877,801 real flows: 0.98 F1 drops to 0.69. Then we found the actual cause - not the lost direction, the lost per-host state. Our sensor keeps 60-second per-host statistics and gets back to 0.981 on the same one-way tap. It runs air-gapped, it abstains when evidence is thin, and every alert is hash-chained so you can prove it."

29.2 1-minute pitch

Add: "There are three kinds of existing solution and each fails for a different reason. Suricata and Zeek need a rule feed, and the update path is exactly the egress path the diode forbids. Commercial NDR needs a cloud callback and costs eight to twenty-two dollars per protected IP per year. Diode vendors move data one way correctly but do not detect threats in it. We built for the constraint instead of around it. And we published the control that narrowed our own claim: sixteen host-window features alone score 0.9812, which is why we say the finding is about host state rather than direction."

29.3 3-minute pitch

Structure: (1) the constraint and why it is becoming common; (2) the measurement - 0.9793 to 0.6903, and the ablation table; (3) the diagnosis - host state, not direction, established by a control that cost us our headline; (4) the sensor - streaming host-window with sketches, six heads, conformal abstention, hash-chained evidence, air-gapped console; (5) what it costs us - 610-672 Mbps rather than 1 Gbps, DGA weak on dictionary families, host-window-alone fails at low density; (6) the discipline - 42 pre-registered predictions, 17 refuted, verdicts printed by the runs, 288 tests, one-command reproduction.

29.4 5-minute technical explanation

Cover, in order: the three PS constraints and their enforcement points; the 103-feature three-way visibility classification and why bidirectional-aggregate features are dropped rather than approximated; the 16 host-window features and the sketch structures behind them, including exact-then-promote at 256 distinct values; the 60-second window with a 15-second lateness budget and the measurement that chose it; split-conformal in Mondrian mode with the measured coverage, abstention and ECE; the evidence chain; the encrypted-traffic path - JA3's failure on Chrome, JA4, ClientHello reassembly at 94%, and QUIC Initial decryption under RFC 9001; then the limits - throughput, sharding non-determinism, DGA, and low-density behaviour.

29.5 Explaining it to a judge

Lead with the measurement, not the architecture. Judges hear proposed systems all day; they rarely hear "we measured the deployment condition first and here is the number". Then volunteer the control - "our own control narrowed our headline claim and it is on the slide" - because volunteering it converts a vulnerability into the credibility of everything else you say. Close on reproducibility: one command, and the verdicts are printed by the run.

29.6 Explaining it to a non-technical audience

"Imagine a secure building where post can be delivered in but nothing can be sent out - not even a request for information. Now imagine the security guard inside can only watch people walking in, never out, and can never phone anyone to ask about a suspicious visitor. Most security systems are designed assuming the guard can do both. We measured how much worse that guard performs - about a third of threats missed - then discovered the fix is not seeing people leave, it is remembering what each visitor has been doing over the last minute. Our system does that, and it works as well as one that can see everything."

29.7 Technical viva

Be ready to open the file. Likely demands: show the CI test that enforces read-only; open the 103-feature column audit; show that source and destination IP are excluded and explain why; show a verdicts JSON and point out a refuted prediction; explain Mondrian conditioning against marginal conformal; explain why exact-then-promote at 256 keeps 14 of 16 features bitwise identical; explain why JA4 sorts and JA3 does not.

30. Expected Questions and Answers

Why not just deploy Suricata or Zeek?

Because the rule-update path is an egress path the diode forbids, and a large part of their rule surface keys on server responses that a one-way tap cannot observe. They are excellent tools in the deployment they were designed for; this is not that deployment.

Why gradient-boosted trees and not deep learning?

Three reasons, in order. Tree ensembles win on tabular network features. They give per-feature importance directly, and explainability is graded. And a deep model in the detection path adds latency and non-determinism that would break the evidence guarantee - the same argument by which we excluded LLMs entirely.

How accurate is it?

0.9810 binary F1 on a one-way tap against 0.9793 with full visibility, on 877,801 real flows with a temporal split. Macro-F1 is much lower - 0.4660 - and that is not a caveat we hide: the temporal split leaves four attack classes absent from training entirely, so macro-F1 partly measures unseen-class generalisation.

Your control scores higher than your sensor. Does that not refute you?

It narrowed our claim, and we published it rather than dropping the control. Host-window features alone score 0.9812. So the honest finding is that host state, not direction, dominates on this corpus. But we then stress-tested that too: at low attack density, host-window-alone collapses from 0.599 to 0.28 macro-F1 while full visibility holds. So the rule is keep per-host state, never run on it alone - which is what the shipped sensor does.

What happens if the model fails or is unsure?

It abstains, explicitly, with a coverage guarantee. Split-conformal in Mondrian mode at alpha = 0.1 gives measured coverage of 0.908 at a 9.11% abstention rate, with 99.55% accuracy on the cases it does answer. The abstention is labelled as insufficient evidence under the unidirectional constraint - the operator knows why.

How does it scale?

Sharded by host across processes: 3.03x on twelve workers, best measured roughly 610-672 Mbps depending on run, against the 178,571 packets per second a 1 Gbps link needs at 700-byte packets. We do not meet line rate. The gap is diagnosed as coordination rather than per-shard compute, and the fix is kernel-level fanout so each shard reads its own NIC queue.

You claimed 1 Gbps earlier. What happened?

We measured it and we were wrong, so we retracted it. That retraction is on the slides. The measured figure is roughly 610-672 Mbps depending on the run - and we should re-run and publish a single number, because our own documents currently disagree by about 10%.

Is the evidence log tamper-proof?

Stronger than tamper-evident, weaker than tamper-proof, and the distinction is worth stating precisely. It is a SHA-256 hash chain, so any edit breaks verification from that point on. But a competent attacker would recompute every hash forward and produce a consistent chain - so each bundle also carries an **HMAC-SHA256** over its canonical form, keyed by a secret the enclave holds. Without that key the forgery fails. What it cannot survive is key compromise, and there is no external anchor or asymmetric signature. Anchoring the head outside the enclave is future work.

Has anyone actually tried to tamper with it?

Yes - seven automated tests do. They edit a value, delete an entry, reorder entries, swap a feature vector, forge the whole chain without the key, and confirm tampering is still caught after a round-trip through JSONL. There is also an API-level test that the verify endpoint detects it, and the console has a tamper control that lets you do it by hand in front of a judge: the browser re-derives every digest itself and then asks the server separately, so you see two independent verifiers agree - and disagree the moment a record is altered.

What is genuinely innovative here?

That the contribution is a measurement rather than an architecture. We characterised what a one-way tap can observe before building anything to exploit it, and the procedure transfers to any constrained sensor placement. The counter-intuitive diagnosis - that the recoverable loss is per-host state rather than direction - is the specific result.

How is this different from existing solutions?

Three categories, three different failure modes: signature IDS needs an egress path for rules; commercial NDR needs a cloud callback and a per-IP licence; diode vendors move data one way but do not detect. We are built for the constraint rather than around it, at zero licence cost and zero egress.

What about encrypted traffic? Surely you cannot see anything.

We never decrypt payload. We read handshakes that are designed to be readable. We initially called QUIC a hard ceiling and were wrong: RFC 9001 derives Initial keys from the connection ID in the clear, so we read those handshakes. That mattered because in our own capture UDP/443 beat TCP/443 by 80,291 to 32,069 packets. We also found JA3 unusable - 92 fingerprints from 97 hellos because Chrome randomises extension order - and switched to JA4, which gives 7.

What are the edge cases?

Three we have measured. Attacks diluted 59:1 to 188:1 inside their own host-window cell defeat host-window features specifically. Dictionary DGA families are close to unusable - suppobox 0.6949 AUC. And below roughly 25 observed benign names, bigram surprisal is near chance.

How would you deploy this at scale?

Honestly: not yet. There is no container image, no service manifest and no packaging. The realistic path is per-segment deployment rather than per-link, kernel-level fanout for throughput, and packaging plus authentication before it goes anywhere less protected than an enclave.

What is the security posture of the tool itself?

No authentication, no authorisation, no accounts. That is a recorded scope decision, not an oversight - it assumes placement inside an already-restricted enclave. If it were exposed on a routable network, anyone reaching it could read every alert. What is enforced is the read-only property: a CI test fails the build if any component opens a socket.

Why is there no database?

Because a database is another service, another dependency and another attack surface inside an enclave that is supposed to have none. State is bounded and in memory - 2.66 MB for 14,817 hosts - and the output is an append-only file that must never be rewritten, which is exactly what the hash chain is designed to detect.

What would you improve with more time?

Packet-size and timing sequence features, because the PS names them and we do not have them - and the reason we recorded for not having them is now obsolete, since we later built the per-packet parser that makes them computable. After that: attack our own evidence chain, re-measure throughput end to end, and test the TLS-novelty channel on real fingerprints.

How do we know these numbers are real?

Each experiment carries predictions written before it ran, and the run prints a verdict for each and writes it to a JSON file. 42 predictions, 17 refuted. A result that only ever confirms its authors is the one to distrust.

Is this production-ready?

No. It is a working prototype with 288 tests and reproducible measurements. It has no packaging, no authentication, no hardening and no operational support. Stating that plainly is more useful to you than a claim we would have to walk back.

31. Demonstration Script

Written so that a teammate who did not build the system can run it. Total time roughly four minutes.

#	Action	What should happen	Point to make
0	Before the room: `cd ekagra/backend` then `python demo.py`. Confirm the console loads.	The read-only API starts on 127.0.0.1:8000 and serves the frontend.	Setup only - do not narrate this.
1	Open the console. Show the summary view.	Alert counts per threat class, the evidence-chain head, and the model hash.	"The model hash is on screen - you know exactly which model produced these alerts."
2	**Disconnect the network** (or state it is already air-gapped) and reload.	The console still works, falling back to the static snapshot.	"No CDN, no framework, no build step. This is the artefact from the repository, running with no network."
3	Filter the alert list to a single threat class, then filter to ABSTAIN.	The list narrows. Abstentions display without a severity.	"An abstention asserts no threat class, so it carries no severity. It used to show HIGH - that was a bug and we fixed it."
4	Open one alert's detail view.	The full feature vector, decision path and contributing packet indices load.	"In an enclave you cannot re-query anything. The alert has to carry its own proof."
5	Call the verification endpoint.	Returns verified true with the entry count and chain head.	"SHA-256 chain across every alert. Edit one record and verification fails from that point on."
6	Open the detector-heads view.	Each head's rationale and its declared known gaps.	"Each head declares what it does not cover. The DGA head says so itself."
7	If asked for proof of the read-only claim: open `tests/test_readonly_constraint.py` and run the suite.	288 tests pass.	"Delete that test and the guarantee is gone. That is the difference between an enforced property and a paragraph."
8	Optional, if time allows: `python experiments/exp04_forward_features.py`	Prints the ablation table and its pre-registered verdicts.	"The verdicts are printed by the run. We cannot quietly move the goalposts."

Table 42. Demonstration sequence.

If the demo breaks

Open `ekagra/frontend/index.html` directly with no server at all. It falls back to the bundled snapshot and everything except the Findings tab and the replay still works. That failure mode is itself the point: the console does not depend on anything.

32. Glossary

Data diode / unidirectional gateway

A hardware device that physically permits data to travel in one direction only. It is the constraint this project is designed around.

One-way tap

A monitoring position that observes only one direction of traffic, with no ability to transmit.

Visibility profile

This project's term for the set of features a given sensor placement can actually compute. Measuring it is the project's central contribution.

Visibility-matched training

Training a model using only the features the deployed sensor will be able to observe.

Host-window aggregate

A per-host statistic computed over a fixed time window - the 16 features that recover detection quality on a one-way tap.

Tumbling window

A non-overlapping fixed-length time window. Here, 60 seconds.

Lateness budget

Extra time a window is held open for out-of-order arrivals before it emits. Here, 15 seconds, chosen by measurement.

HyperLogLog (HLL)

A probabilistic structure estimating the number of distinct items in near-constant memory. Used for per-host fan-out.

Count-Min Sketch (CMS)

A probabilistic structure estimating item frequencies in sublinear memory. Used for source-IP distribution statistics.

Misra-Gries

A deterministic heavy-hitters algorithm identifying the most frequent items in bounded memory.

Conformal prediction

A framework producing prediction sets with a finite-sample coverage guarantee, by comparing a new score against calibration-set quantiles.

Mondrian conformal

Conformal prediction where quantiles are computed per class, so a rare class cannot have its coverage absorbed by a common one.

Abstention

An explicit refusal to predict, emitted when the calibrated prediction set is not a singleton. A first-class output, not a failure.

Expected Calibration Error (ECE)

The average gap between stated confidence and observed accuracy. Measured here at 0.00084.

Binary F1 / macro F1

Binary F1 is the harmonic mean of precision and recall for attack-versus-benign. Macro F1 averages per-class F1 equally, so rare and unseen classes weigh as heavily as common ones.

Temporal split

Train on earlier data, test on later. Mandatory for time-series security data; a random split leaks the same attack episode into both sides.

JA3 / JA3S / JA4

TLS fingerprints. JA3 is an MD5 over the client hello's version, ciphers, extensions, curves and point formats in wire order; JA3S is the server equivalent; JA4 sorts ciphers and extensions, which makes it stable against Chrome's extension-order randomisation.

GREASE (RFC 8701)

Deliberately random values TLS clients insert to keep implementations tolerant. They must be stripped before fingerprinting or every handshake looks unique.

QUIC Initial decryption (RFC 9001)

QUIC's first-flight packets are encrypted with keys derived from the Destination Connection ID, which is in the clear, using a published salt. This makes the ClientHello readable without any secret - it is not payload decryption.

DGA

Domain Generation Algorithm - malware generating many candidate C2 domain names. Families differ enormously: uniform-random families are easy to spot, dictionary-word families are not.

Bigram surprisal

The improbability of a string's character pairs under a model of normal domain names. The strongest single DGA feature here, but it needs roughly 25 observed benign names to warm up.

Evidence bundle / hash chain

A per-alert record containing packet indices, feature vector, model hash and decision path, linked by SHA-256 to its predecessor so edits are detectable.

Pre-registered prediction

A hypothesis written into the experiment file before the first run. The run prints the verdict. Never edited afterwards.

MITRE ATT&CK

A public taxonomy of adversary tactics and techniques, used here for threat-stage vocabulary.

NTRO / NCIIPC / CERT-In

India's National Technical Research Organisation (the client for this problem statement); the National Critical Information Infrastructure Protection Centre; and the Indian Computer Emergency Response Team.

33. One-Page Cheat Sheet

Project	EKAGRA - SIH26145 - NTRO - Team AlgoRhythms
Problem	A data diode gives the sensor no return path. Published detectors assume both directions plus a reachable threat-intel service, and collapse from 0.9793 to 0.6903 binary F1 when deployed on a one-way tap.
Solution	Measure the visibility profile of the tap, then build a sensor to it: streaming 60-second per-host aggregates recover 0.6903 to 0.9810.
Target users	NTRO and NCIIPC-designated Critical Information Infrastructure operators; analysts inside diode-protected enclaves.
Core features	Visibility-matched training; streaming host-window aggregation; six detector heads; conformal abstention; HMAC-keyed hash-chained evidence; JA4 and QUIC handling; an air-gapped console.
Architecture	Source adapters -> visibility filter -> flow assembler -> streaming host-window -> detector heads -> conformal calibration -> evidence bundle -> read-only API and console. One-way, CI-enforced.
Tech stack	Python 3.11+, XGBoost, NumPy/pandas, scikit-learn, SciPy, cryptography, pytest. Frontend: plain HTML/CSS/JS, no framework, no CDN, no build.
AI/ML	GBDT per detector head over 35 forward-observable plus 16 engineered host-window features; split-conformal calibration in Mondrian mode at alpha = 0.1.
Key innovation	The contribution is a measurement, not an architecture - and the diagnosis is counter-intuitive: the recoverable loss is per-host state, not direction.
Headline results	0.9810 binary F1 on a one-way tap vs 0.9793 full visibility on 877,801 real flows. Streaming matches batch to 0.00001. 2.66 MB for 14,817 hosts. Coverage 0.908 at 9.11% abstention. 288 tests. 17 of 42 predictions refuted.
Advantages	Zero licence, zero egress, sovereign, abstains rather than guesses, every alert provable, bounded memory.
Limitations	610-672 Mbps, not 1 Gbps. DGA weak on dictionary families (suppobox 0.6949). Host-window-alone fails at low attack density. No auth, no packaging. Packet-size and timing sequences - a PS requirement - not implemented.
Future scope	Short: sequence features, attack the evidence chain, re-measure throughput. Medium: kernel fanout, real-corpus packet validation, authentication. Long: 10 Gbps, external chain anchoring.
One-line pitch	*"We measured what a one-way tap costs a threat detector, found the loss was per-host state rather than direction, and built the sensor that recovers it - air-gapped, abstaining, and provable."*